Selenium Assignment

Web Automation & Scraping Report

Name: JANNATUL FERDAUS OISHI

Student ID: 2022-3-60-216

Course: CSE430

Date: 19 April, 2026

Target Website: https://books.toscrape.com

Table of Contents

1. Task 1 - Browser Navigation & Assertions 3
2. Task 2 - Element Locators (4 Strategies) 4
3. Task 3 - Scraping Pages 1-4 & CSV Export 6
4. Task 4 - Book Detail Validation Tests 8
5. Test Summary & Conclusion 9

Task 1: Browser Navigation & Basic Assertions

Objective

Navigate to https://books.toscrape.com using Selenium WebDriver, verify the page title and URL, maximize the browser window, and confirm that the page state remains consistent after a refresh.

Approach

A Chrome WebDriver instance was initialized using selenium.webdriver.Chrome(). The driver navigated to the target URL using driver.get(). The page title and current URL were captured using driver.title and driver.current_url respectively. The browser window was maximized with driver.maximize_window().

Key Code Snippet

```
driver = webdriver.Chrome()
driver.get("https://books.toscrape.com")
title = driver.title
url = driver.current_url
driver.maximize_window()
```

Assertions Performed

> PASS: 'Books' found in page title
> PASS: URL starts with https://
> PASS: Title unchanged after page refresh

Refresh Verification Code

```
old_title = driver.title
driver.refresh()
time.sleep(2)
new_title = driver.title
assert old_title == new_title
```

All three assertions passed successfully. The page title contained 'Books', the URL used HTTPS protocol, and the title remained identical after refreshing the page.

Task 2: Element Locator Strategies

Objective

Demonstrate four different Selenium locator strategies (CLASS_NAME, CSS_SELECTOR, XPATH, and TAG_NAME) to find various elements on the page.

a) By.CLASS_NAME - Finding Book Elements

Used By.CLASS_NAME with 'product_pod' to locate all book containers on the page. Each product_pod div contains a book's thumbnail, title, price, and rating.

```
books = driver.find_elements(By.CLASS_NAME, "product_pod")
print("Total books found:", len(books))  # 20 per page
```

> PASS: Books found using CLASS_NAME (20 elements)

b) By.CSS_SELECTOR - Finding Star Ratings

Used By.CSS_SELECTOR with '.star-rating' to find all star rating elements. The CSS selector targets elements with the class 'star-rating'.

```
ratings = driver.find_elements(By.CSS_SELECTOR, ".star-rating")
print("Sample class:", ratings[0].get_attribute("class"))
```

> PASS: Ratings found using CSS_SELECTOR

c) By.XPATH - Finding Next Button

Used By.XPATH to locate the 'Next' pagination button. The XPath expression targets an anchor tag inside an li element with class='next'.

```
next_btn = driver.find_elements(
    By.XPATH, "//li[@class='next']/a"
)
```

> PASS: Next button found using XPATH

d) By.TAG_NAME - Finding Article Elements

Used By.TAG_NAME with 'article' to find all article HTML elements on the page. Each book is wrapped in an <article> tag.

```
articles = driver.find_elements(By.TAG_NAME, "article")
print("Total articles:", len(articles))
```

> PASS: Articles found using TAG_NAME

All four locator strategies successfully identified the expected elements. CLASS_NAME and TAG_NAME found 20 elements each (one per book), CSS_SELECTOR found 20 star-rating elements, and XPATH located the pagination button.

Task 3: Scraping Pages 1-4 & CSV Export

Objective

Scrape book data (title, price, rating, availability) from the first 4 pages of the website, navigate between pages using the 'Next' button, and export all collected data to a CSV file.

Approach

A loop iterated through pages 1 to 4. On each page, all book elements were located using By.CLASS_NAME 'product_pod'. For each book, the following data was extracted:

- Title: from the <a> tag's 'title' attribute inside <h3> (full, non-truncated title)
- Price: from the element with class 'price_color'
- Availability: from the element with class 'availability'
- Rating: extracted from the second CSS class of the 'star-rating' element

Page Navigation & Data Extraction Code

```
for page in range(1, 5):
    books = driver.find_elements(By.CLASS_NAME, "product_pod")
    for book in books:
        title = book.find_element(By.TAG_NAME, "h3")\
            .find_element(By.TAG_NAME, "a")\
            .get_attribute("title")
        price = book.find_element(
            By.CLASS_NAME, "price_color").text
        rating = book.find_element(
            By.CLASS_NAME, "star-rating")\
            .get_attribute("class").split()[1]
        books_data.append({...})
    # Navigate to next page
    driver.find_element(
        By.XPATH, "//li[@class='next']/a").click()
```

CSV Export

The collected data was saved to 'books_data.csv' using Python's csv.DictWriter with columns: page, title, price, rating, and availability.

```
with open("books_data.csv", "w", newline="",
          encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=[
        "page","title","price","rating","availability"])
    writer.writeheader()
    writer.writerows(books_data)
```

Results

> PASS: Total 80 books collected (20 per page x 4 pages)

> PASS: CSV file saved successfully as books_data.csv

Sample data from the first 3 books collected:

Pg	Title	Price	Rating	Availability
1	A Light in the Attic	51.77	Three	In stock
1	Tipping the Velvet	53.74	One	In stock
1	Soumission	50.10	One	In stock

Task 4: Book Detail Page Validation

Objective

Navigate to individual book detail pages for the first 3 scraped books and validate that the displayed information matches the data collected during scraping. Multiple assertions were performed on each book's detail page.

Approach

For each of the first 3 books, the script clicked on the book link using By.PARTIAL_LINK_TEXT, navigated to the detail page, and performed five validation checks. After each book, driver.back() was used to return to the listing page.

Validation Code

```
for book in sample_books:
    link = driver.find_element(
        By.PARTIAL_LINK_TEXT, title[:10])
    link.click()
    # a. Page title not empty
    log("Page title not empty",
        driver.title != "")
    # b. Title matches
    page_title = driver.find_element(
        By.TAG_NAME, "h1").text
    log("Title matches",
        title[:10] in page_title)
    # c. Price matches
    page_price = driver.find_element(
        By.CLASS_NAME, "price_color").text
    log("Price matches",
        page_price == book["price"])
    # d. Add to basket button enabled
    btn = driver.find_element(By.CSS_SELECTOR,
        "button.btn-primary[type='submit']")
    log("Add to basket enabled",
        btn.is_enabled())
    # e. Availability check
    avail = driver.find_element(
        By.CLASS_NAME, "availability").text
    log("In stock check",
        "In stock" in avail)
    driver.back()
```

Tests Per Book (5 assertions each)

Book: A Light in the Attic

> PASS: Page title is not empty

> PASS: Book title matches scraped title

> PASS: Price matches scraped price
> PASS: Add to basket button is enabled
> PASS: Availability shows 'In stock'

Book: Tipping the Velvet

> PASS: Page title is not empty
> PASS: Book title matches scraped title
> PASS: Price matches scraped price
> PASS: Add to basket button is enabled
> PASS: Availability shows 'In stock'

Book: Soumission

> PASS: Page title is not empty
> PASS: Book title matches scraped title
> PASS: Price matches scraped price
> PASS: Add to basket button is enabled
> PASS: Availability shows 'In stock'

Test Summary & Conclusion

Overall Test Results

Task	Tests	Status
Task 1: Navigation & Assertions	3	All Passed
Task 2: Element Locators	4	All Passed
Task 3: Scraping & CSV Export	2	All Passed
Task 4: Detail Page Validation	15	All Passed
Total	**24**	**24/24 Passed**

Conclusion

All tasks were successfully completed using Selenium WebDriver with Python. The assignment demonstrated proficiency in:

- Browser automation and navigation using Selenium WebDriver
- Four different element locator strategies (CLASS_NAME, CSS_SELECTOR, XPATH, TAG_NAME)
- Multi-page web scraping with pagination handling
- Data extraction and CSV file export using Python's csv module
- Automated testing with assertions on book detail pages
- Page navigation using driver.back() and PARTIAL_LINK_TEXT for clicking links

A total of 80 books were scraped from 4 pages and saved to books_data.csv. All 24 assertions across all tasks passed successfully, confirming that the website's content is consistent and the automation scripts work correctly.

Tools & Technologies Used

- Python 3
- Selenium WebDriver (Chrome)
- Jupyter Notebook
- CSV module for data export